

# Generative Models for Cat Image Synthesis

Deep Learning – Project III

Anna Ostrowska

June 2026

## Abstract

This report presents the implementation and evaluation of three generative modelling approaches for synthesising cat images at  $64 \times 64$  resolution: a Deep Convolutional GAN trained with Wasserstein loss and gradient penalty (DCGAN/WGAN-GP), a Vector-Quantized Variational Autoencoder with an autoregressive PixelCNN prior (VQ-VAE), and a Denoising Diffusion Probabilistic Model with DDIM fast sampling (DDPM). All models are trained from scratch on the CAT Dataset (9 997 images) using a single consumer GPU with 6 GiB VRAM. Models are compared quantitatively using the Fréchet Inception Distance (FID) computed on 5 000 generated images and qualitatively through visual inspection and latent space interpolation. DDPM achieves the lowest FID (24.11 at 50 DDIM steps with EMA weights), substantially outperforming VQ-VAE (187.54) and DCGAN (221.51). A post-hoc analysis of the DCGAN training configuration reveals that generator starvation caused by  $n_{\text{critic}} = 5$  likely contributed to the high FID; corrected configurations reduce FID to 98.99 in the best single run (standard DCGAN) and 158.48 (WGAN-GP  $n_c = 2$ ). Ablation studies investigate the effect of DDIM inference steps and EMA weights on DDPM quality, and the sensitivity of DCGAN to latent dimension and learning rate. An exploratory extension trains DCGAN on a combined cats-and-dogs dataset, finding a suggestive improvement in cat FID that warrants further investigation. Source code is provided; model checkpoints and generated outputs are submitted separately.

## Contents

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                | <b>3</b> |
| <b>2</b> | <b>Background and Related Work</b>                 | <b>3</b> |
| 2.1      | Generative Adversarial Networks . . . . .          | 3        |
| 2.2      | Variational Autoencoders and VQ-VAE . . . . .      | 4        |
| 2.3      | Denoising Diffusion Probabilistic Models . . . . . | 4        |
| 2.4      | Fréchet Inception Distance . . . . .               | 5        |
| <b>3</b> | <b>Dataset and Preprocessing</b>                   | <b>5</b> |
| 3.1      | Cat Dataset . . . . .                              | 5        |
| 3.2      | Preprocessing Pipeline . . . . .                   | 5        |
| 3.3      | Data Augmentation . . . . .                        | 5        |

|           |                                                      |           |
|-----------|------------------------------------------------------|-----------|
| <b>4</b>  | <b>Methods</b>                                       | <b>6</b>  |
| 4.1       | DCGAN with WGAN-GP . . . . .                         | 6         |
| 4.2       | VQ-VAE + PixelCNN Prior . . . . .                    | 6         |
| 4.3       | DDPM with DDIM Sampling . . . . .                    | 7         |
| <b>5</b>  | <b>Computational Considerations</b>                  | <b>7</b>  |
| <b>6</b>  | <b>Results</b>                                       | <b>8</b>  |
| 6.1       | FID Comparison . . . . .                             | 8         |
| 6.2       | Generated Samples . . . . .                          | 9         |
| 6.3       | Training Dynamics . . . . .                          | 9         |
| 6.4       | Sample Progression . . . . .                         | 10        |
| 6.5       | VQ-VAE Reconstruction . . . . .                      | 11        |
| 6.6       | Latent Interpolation . . . . .                       | 11        |
| <b>7</b>  | <b>Mode Collapse Analysis</b>                        | <b>13</b> |
| <b>8</b>  | <b>DCGAN Training Bottleneck Analysis</b>            | <b>13</b> |
| 8.1       | Generator Starvation . . . . .                       | 13        |
| 8.2       | Corrected Configurations . . . . .                   | 14        |
| 8.3       | Discussion . . . . .                                 | 14        |
| <b>9</b>  | <b>Ablation Studies</b>                              | <b>15</b> |
| 9.1       | DDPM: DDIM Inference Steps and EMA Weights . . . . . | 15        |
| 9.2       | VQ-VAE: Codebook Size . . . . .                      | 15        |
| 9.3       | DCGAN: Latent Dimension and Learning Rate . . . . .  | 16        |
| <b>10</b> | <b>Cats and Dogs Experiment</b>                      | <b>17</b> |
| 10.1      | Setup . . . . .                                      | 17        |
| 10.2      | Evaluation . . . . .                                 | 17        |
| 10.3      | Qualitative Assessment . . . . .                     | 18        |
| 10.4      | Quantitative Discussion . . . . .                    | 18        |
| <b>11</b> | <b>Conclusion</b>                                    | <b>19</b> |

# 1 Introduction

This report presents the design, training, and evaluation of three classes of generative image models applied to a dataset of cat photographs. The three approaches investigated are:

1. **DCGAN** with Wasserstein loss and gradient penalty (WGAN-GP),
2. **VQ-VAE** with an autoregressive PixelCNN prior,
3. **DDPM** (Denoising Diffusion Probabilistic Model) with DDIM sampling.

Models are compared quantitatively using the Fréchet Inception Distance (FID) and qualitatively through visual inspection of generated samples and latent interpolation strips. Additionally, an ablation study is conducted for each model class, and a supplementary experiment trains a DCGAN on a combined cats-and-dogs dataset.

**Deviations from the project plan.** Three aspects of the implementation diverged from the original project plan (KM0, submitted 19 May 2026):

1. **Label smoothing.** The plan stated “WGAN-GP loss with one-sided label smoothing (real labels = 0.9) is used for DCGAN.” In practice, this parameter – present in the shared codebase for standard BCE-GAN variants – operates on binary cross-entropy targets and has no effect on the WGAN-GP loss path, which uses an unbounded scalar critic score rather than class probabilities. Label smoothing was therefore inactive during WGAN-GP training.
2. **Latent interpolation.** The plan described linear interpolation between latent vectors. Linear interpolation is used as the primary method throughout. Spherical linear interpolation (slerp) is additionally shown for comparison for DCGAN and DDPM (Section 6.6): it avoids the low-density region near the origin that linear interpolation traverses in  $\mathcal{N}(0, I)$  space, which produces blurry intermediate frames. VQ-VAE uses linear interpolation only, as the prior operates over discrete tokens rather than a Gaussian.
3. **DCGAN ablation depth.** The plan specified 50-epoch ablation runs with a single seed. In practice, ablations were repeated across three random seeds (0, 42, 123) to quantify seed variance, keeping 50 epochs per run as planned. Results are reported as mean  $\pm$  population standard deviation in Table 4.

## 2 Background and Related Work

### 2.1 Generative Adversarial Networks

Generative Adversarial Networks (GANs) [3] frame image synthesis as a two-player game between a Generator  $G$  and a Discriminator  $D$ . The generator maps samples  $\mathbf{z} \sim p(\mathbf{z})$  from a simple prior (typically  $\mathcal{N}(0, I)$ ) to synthetic images, while the discriminator attempts to distinguish real from generated data. Training minimises the standard min-max objective:

$$\min_G \max_D \mathbb{E}_{x \sim p_r} [\log D(x)] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))] \quad (1)$$

GANs are prone to *mode collapse* – a failure mode where the generator produces only a small subset of the data distribution – and to unstable gradient dynamics. The Deep

Convolutional GAN (DCGAN) [10] introduced architectural guidelines (all-convolutional networks, batch normalisation, LeakyReLU activations) that substantially improved training stability.

**WGAN and WGAN-GP.** Arjovsky et al. [1] proposed replacing the Jensen-Shannon divergence objective with the Wasserstein-1 distance, which provides smoother gradients and alleviates mode collapse. However, their weight-clipping approach introduces gradient pathologies. Gulrajani et al. [4] resolved this by enforcing the Lipschitz constraint via a gradient penalty term on interpolated samples, yielding the Wasserstein GAN with Gradient Penalty (WGAN-GP). Crucially, the WGAN-GP critic outputs an unbounded scalar score rather than a probability, so binary label targets such as  $\{0, 1\}$  or their smoothed variants are not used in the Wasserstein loss path.

## 2.2 Variational Autoencoders and VQ-VAE

Variational Autoencoders (VAEs) [9] train an encoder–decoder pair by maximising the evidence lower bound (ELBO):

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \| p(z)), \quad (2)$$

where  $q_\phi$  is the approximate posterior and  $p_\theta$  the decoder. The continuous latent space allows smooth interpolation but tends to produce blurry reconstructions.

The Vector-Quantized VAE (VQ-VAE) [14] replaces the continuous latent code with a discrete codebook. The encoder output is replaced by the nearest codebook entry (straight-through gradient); codebook updates use exponential moving averages (EMA) for stability. Because the prior  $p(z)$  is non-trivial (discrete, structured), a separate autoregressive model (PixelCNN [13]) is trained over the quantised token sequences after the autoencoder converges.

## 2.3 Denoising Diffusion Probabilistic Models

DDPM [6] defines a *forward process* that gradually adds Gaussian noise to data:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I), \quad (3)$$

and a learnable *reverse process*  $p_\theta(x_{t-1}|x_t)$  implemented as a U-Net that predicts the noise  $\epsilon$  added at each step. The training objective is:

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]. \quad (4)$$

Dhariwal and Nichol [2] demonstrated that diffusion models with classifier guidance surpass GANs on FID benchmarks.

**DDIM.** Song et al. [12] showed that the same trained model can be used with a deterministic, non-Markovian sampling process (Denoising Diffusion Implicit Models) requiring as few as 20–50 function evaluations rather than the full  $T = 1000$  steps, at negligible quality loss.



## 2.4 Fréchet Inception Distance

FID [5] measures the Fréchet (Wasserstein-2) distance between the distributions of real and generated images in the feature space of an Inception-v3 network:

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}), \quad (5)$$

where  $(\mu_r, \Sigma_r)$  and  $(\mu_g, \Sigma_g)$  are the mean and covariance of the Inception activations for real and generated sets. Lower FID indicates closer distributional similarity. We use `torch-fidelity` with 5 000 real and 5 000 generated images.

## 3 Dataset and Preprocessing

### 3.1 Cat Dataset

The primary dataset is the CAT Dataset [15], consisting of 9 997 RGB cat photographs with heterogeneous resolutions, organised in subdirectories `CAT_00` through `CAT_06`.

### 3.2 Preprocessing Pipeline

A uniform preprocessing pipeline was applied to all images:

1. **Resize** the shorter edge to 64 pixels.
2. **CenterCrop** to  $64 \times 64$ .
3. **Normalize** to  $[-1, 1]$  via  $x \leftarrow (x - 0.5)/0.5$  per channel.

All models operate on  $64 \times 64 \times 3$  tensors. Figure 1 shows a random selection of preprocessed images. A fixed random seed (42) is used for all experiments. DataLoaders use `drop_last=True`, giving  $\lfloor 9997/64 \rfloor = 156$  batches per epoch.



Figure 1: Random sample of real cat images after  $64 \times 64$  center-crop preprocessing.

### 3.3 Data Augmentation

An optional augmentation pipeline was implemented. When enabled, the following transforms are applied before normalization:

- `RandomResizedCrop` (scale 0.85–1.0),

- `RandomHorizontalFlip` ( $p = 0.5$ ),
- `ColorJitter` (brightness/contrast/saturation = 0.2, hue = 0.05),
- `RandomGrayscale` ( $p = 0.05$ ),
- `RandomRotation` ( $\pm 10^\circ$ ),
- `RandomErasing` ( $p = 0.3$ , scale 2–8%).

The baseline models are trained without augmentation; augmentation effects are investigated in the ablation study (Section 9).

## 4 Methods

### 4.1 DCGAN with WGAN-GP

**Architecture.** The Generator  $G$  maps a latent vector  $\mathbf{z} \sim \mathcal{N}(0, I)$ ,  $\mathbf{z} \in \mathbb{R}^{128}$ , to a  $64 \times 64 \times 3$  image through four transposed-convolution blocks (BatchNorm + ReLU), ending in a Tanh activation. The Discriminator  $D$  mirrors this with four strided-convolution blocks (LeakyReLU,  $\alpha = 0.2$ ) and outputs a scalar logit without sigmoid. The architecture follows DCGAN guidelines [10].

**Implementation note.** The original discriminator used `BatchNorm2d`, which is a sub-optimal choice for WGAN-GP: BatchNorm mixes batch statistics and interferes with the per-sample gradient penalty computation. The reported baseline FID of 221.51 was obtained with this BatchNorm discriminator. The code has since been corrected to `InstanceNorm2d(affine=True)`; results from the corrected critic are reported separately in Section 8.

**Training objective.** We adopt the Wasserstein loss with gradient penalty (WGAN-GP [4]):

$$\mathcal{L}_D = \mathbb{E}_{\tilde{x} \sim P_g}[D(\tilde{x})] - \mathbb{E}_{x \sim P_r}[D(x)] + \lambda \mathbb{E}_{\hat{x}}[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2], \quad (6)$$

with  $\lambda = 10$  and  $\hat{x}$  sampled uniformly along straight lines between real and generated data. The discriminator (critic) is updated  $n_{\text{critic}} = 5$  times per generator step. Because the WGAN-GP critic returns an unbounded scalar rather than a class probability, binary label targets – and any smoothing thereof – have no role in this loss formulation and were not applied.

**Hyperparameters.** Trained for 100 epochs, batch size 64, Adam ( $\beta_1 = 0.5$ ,  $\beta_2 = 0.999$ ), learning rate  $2 \times 10^{-4}$  for both  $G$  and  $D$ .

### 4.2 VQ-VAE + PixelCNN Prior

**Architecture.** The VQ-VAE [14] consists of an encoder  $E$  that maps a  $64 \times 64$  image to a  $16 \times 16$  discrete codebook index map, a vector-quantization layer with codebook size  $K = 512$  and embedding dimension  $d = 64$ , and a decoder  $D$  that reconstructs the image from quantized embeddings. The encoder and decoder each use strided-convolution and transposed-convolution blocks with two residual layers. Codebook updates use exponential moving averages (EMA) with decay  $\gamma = 0.99$ .

Two-phase training:

1. **Phase 1 – Autoencoder** (100 epochs): minimizes reconstruction loss + VQ commitment loss  $\beta \|z_e - \text{sg}(z_q)\|^2$ ,  $\beta = 0.25$ .
2. **Phase 2 – Prior** (100 epochs): a PixelCNN [13] with 15 masked residual layers (128 filters each) is trained autoregressively on the flattened  $16 \times 16 = 256$  token sequences from the frozen encoder.

**Sampling.** At inference, the PixelCNN samples a 256-token sequence autoregressively; tokens are mapped to embeddings via the codebook and decoded by  $D$ .

### 4.3 DDPM with DDIM Sampling

**Architecture.** We implement the DDPM framework [6] with a U-Net backbone. The U-Net has channel multipliers  $(1, 2, 2, 2)$ , two residual blocks per resolution, and self-attention at the  $16 \times 16$  feature map level. An exponential moving average (EMA, decay 0.9999) of model weights is maintained and used at inference.

**Diffusion schedule.** Linear noise schedule with  $T = 1000$  steps,  $\beta_1 = 10^{-4}$ ,  $\beta_T = 0.02$ . The forward process is:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I), \quad (7)$$

where  $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$ . The model predicts the added noise  $\epsilon_\theta(x_t, t)$  and is trained with simple  $\ell_2$  loss.

**DDIM Sampling.** Deterministic DDIM [12] allows generating samples with far fewer function evaluations than the full  $T = 1000$  reverse chain. At inference we use 50 DDIM steps ( $\eta = 0.0$ ) by default, giving a  $20\times$  speedup.

**Hyperparameters.** Trained for 200 epochs, batch size 16 (limited by 6 GiB VRAM), Adam ( $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ), learning rate  $2 \times 10^{-4}$  with linear warmup.

## 5 Computational Considerations

All experiments were run on a single NVIDIA GPU with 6 GiB VRAM and a Windows host. Key adaptations:

- `num_workers=0` in all DataLoaders (Windows multiprocessing limitation).
- DDPM batch size reduced to 16 to fit within VRAM.
- DDIM sampling used instead of full reverse diffusion to cut inference time from  $\approx 8$  h (1000 steps) to  $\approx 25$  min (50 steps).
- VQ-VAE EMA codebook uses `scatter_add_ + bincount` instead of `F.one_hot` to avoid out-of-memory issues.
- Global seed 42 for reproducibility.

## 6 Results

### 6.1 FID Comparison

Fréchet Inception Distance (FID) [5] measures the distance between the distribution of real images and generated images in the feature space of an Inception-v3 network. We generate 5 000 samples per model and compare against 5 000 real images drawn from the training set.

Table 1: FID scores on the cat dataset ( $64 \times 64$ , 5 000 samples). All results are single-run. The DCGAN entry uses the original training configuration later found to be under-trained (see Section 8); a corrected standard DCGAN achieves FID 98.99.

| Model                                            | FID ↓        | Notes                                              |
|--------------------------------------------------|--------------|----------------------------------------------------|
| DCGAN (WGAN-GP, $n_c=5$ )                        | 221.51       | 100 ep., 3 100 G-steps; under-trained <sup>†</sup> |
| DCGAN (standard, corrected)                      | 98.99        | 150 ep., $d_{\text{ch}} = 128$ ; single run        |
| 3-seed ( $d_{\text{ch}} = 64$ ): $142.6 \pm 3.4$ |              | see §8                                             |
| VQ-VAE + PixelCNN                                | 187.54       | 100 + 100 epochs                                   |
| DDPM (50 DDIM steps, EMA)                        | <b>24.11</b> | 200 epochs                                         |

<sup>†</sup> With  $n_c=5$  the generator receives only 3 100 weight updates over 100 epochs vs. 15 600 for a standard 1:1 update ratio.

DDPM achieves a substantially lower FID, consistent with the literature showing diffusion models outperforming GANs and VAEs on small-scale datasets [2]. The DCGAN result depends strongly on training configuration: the original WGAN-GP setting ( $n_c = 5$ ) results in only 3 100 generator updates over 100 epochs, which we show in Section 8 is a likely source of under-training. A corrected standard DCGAN achieves FID 98.99, substantially narrowing the gap with VQ-VAE.

**Scope of this comparison.** Two caveats apply. First, the comparison is not compute-normalised: DDPM was trained for  $\approx 12$  h, DCGAN for  $\approx 2$  h, and VQ-VAE for  $\approx 4$  h (approximate wall-clock times observed manually on an NVIDIA RTX 4050 Laptop GPU). The table reflects the best feasible configuration for each model under the 6 GiB VRAM constraint, not a strictly equal-budget comparison. Second, FID was computed against a subset of the *training* distribution; no held-out test set was used. The metric therefore measures similarity to available training data rather than generalisation to unseen cat images. Given that the main comparisons are single-run, the large gap between DDPM and the other models is reliably qualitative, while smaller differences should not be over-interpreted.

## 6.2 Generated Samples

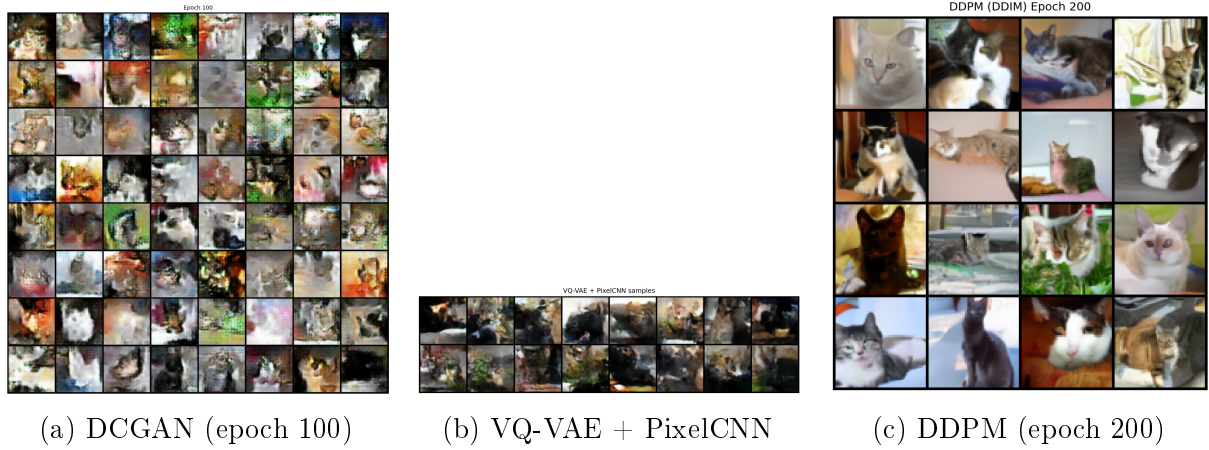


Figure 2: Generated samples from each model at the end of training.

DDPM samples show the most coherent cat structure with natural fur textures. DCGAN samples exhibit some diversity but with noticeable artifacts and occasional mode collapse symptoms (similar face structures). VQ-VAE samples are globally plausible but blurrier, which likely reflects limitations of the PixelCNN prior rather than the autoencoder itself (see Section 6.5).

## 6.3 Training Dynamics

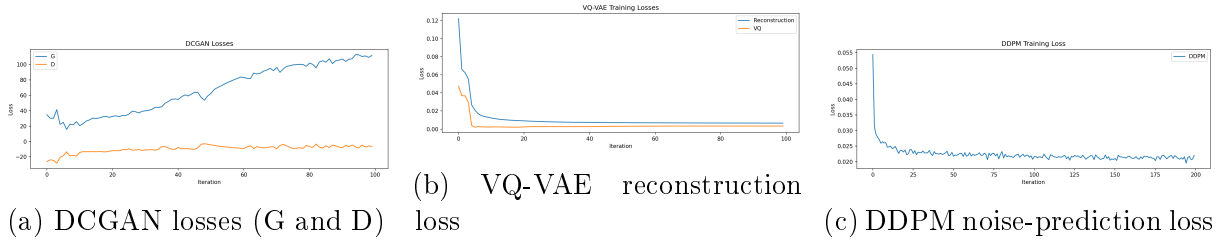


Figure 3: Training loss curves for all three models.

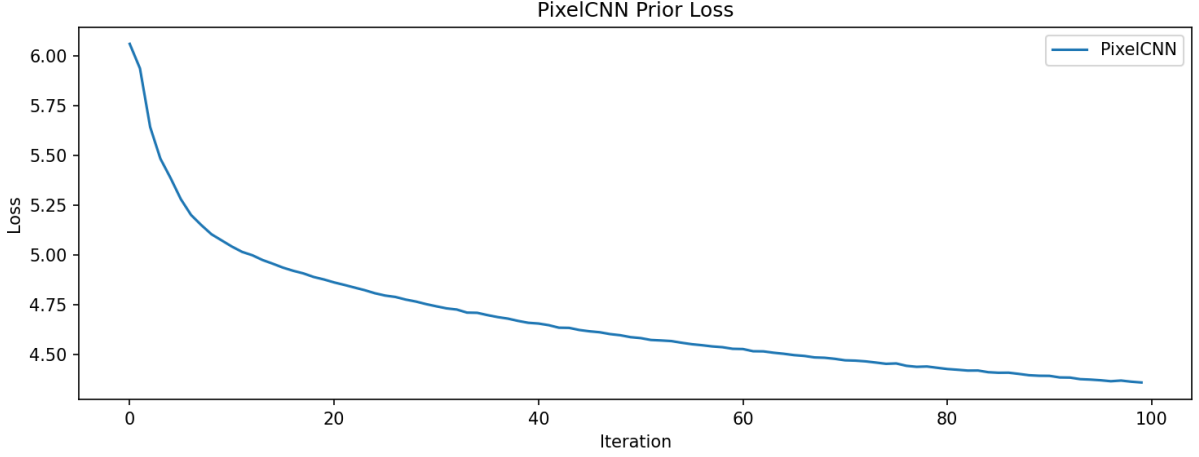


Figure 4: VQ-VAE PixelCNN prior training loss (Phase 2, 100 epochs).

DCGAN training shows the characteristic adversarial oscillation: generator and discriminator losses compete without a stable equilibrium. VQ-VAE reconstruction loss decreases steadily throughout Phase 1 and the PixelCNN prior loss decreases more slowly in Phase 2. DDPM loss decreases monotonically and smoothly, reflecting the well-defined mean-squared-error noise-prediction objective.

All models were trained on the full dataset without a validation split; early stopping was therefore not applied. Sample grids saved at regular checkpoints (shown in Section 6.4) serve as the primary qualitative proxy for monitoring training progress and detecting mode collapse. Per-epoch FID evaluation was not feasible due to its computational cost ( $\approx 5$  min per evaluation).

## 6.4 Sample Progression



Figure 5: DCGAN sample grids at training epochs 10, 50, and 100.



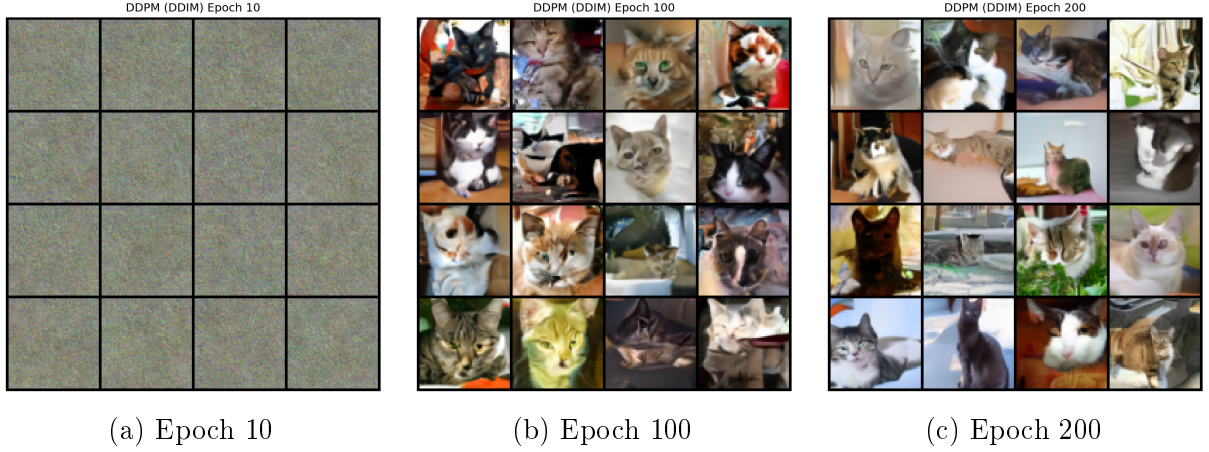


Figure 6: DDPM sample grids at training epochs 10, 100, and 200.

## 6.5 VQ-VAE Reconstruction



Figure 7: VQ-VAE reconstructions at epoch 100 (top row: inputs, bottom row: reconstructed).

The VQ-VAE achieves a codebook perplexity of 403.3/512, indicating that  $\approx 79\%$  of codes are actively used. Reconstruction loss at convergence is 0.0066 (MSE), VQ commitment loss 0.0033. The good reconstruction quality contrasts with the blurrier unconditional samples, which suggests that the PixelCNN prior is a likely bottleneck: the autoencoder itself captures sufficient detail, but the prior’s ability to model the full distribution of token sequences is limited.

## 6.6 Latent Interpolation

To assess the smoothness of the learned latent space, we interpolate linearly between two latent vectors  $\mathbf{z}_A$  and  $\mathbf{z}_B$ :

$$\mathbf{z}_\alpha = (1 - \alpha) \mathbf{z}_A + \alpha \mathbf{z}_B, \quad \alpha \in \left\{0, \frac{1}{9}, \dots, 1\right\}. \quad (8)$$

For VQ-VAE, interpolation is performed between codebook embedding maps sampled from the PixelCNN prior. For DCGAN and DDPM, whose priors are  $\mathcal{N}(0, I)$ , we additionally show spherical linear interpolation (slerp):

$$\mathbf{z}_\alpha = \frac{\sin((1-\alpha)\theta)}{\sin\theta} \mathbf{z}_A + \frac{\sin(\alpha\theta)}{\sin\theta} \mathbf{z}_B, \quad \theta = \arccos\left(\frac{\mathbf{z}_A \cdot \mathbf{z}_B}{\|\mathbf{z}_A\| \|\mathbf{z}_B\|}\right). \quad (9)$$



(a) DCGAN: linear interpolation (top row) vs. slerp (bottom row) in  $\mathcal{N}(0, I)$  latent space.



(b) VQ-VAE: linear interpolation between PixelCNN-sampled codebook embedding maps.



(c) DDPM: linear interpolation (top row) vs. slerp (bottom row) between initial noise tensors  $x_T \sim \mathcal{N}(0, I)$ .

Figure 8: Latent interpolation strips (10 images: 2 endpoints + 8 intermediate frames). Linear interpolation is the primary method. Slerp is shown as a comparison for DCGAN and DDPM.

**Discussion.** Linear interpolation is used as the primary method throughout. The comparison with slerp for DCGAN and DDPM illustrates a known artefact of Gaussian priors: the midpoint of a linear interpolation between two vectors drawn from  $\mathcal{N}(0, I)$  has expected norm  $\|\frac{1}{2}(z_0 + z_1)\| \approx \frac{1}{\sqrt{2}}\|z\|$ , placing intermediate latents in a low-density region near the origin. The resulting decoded images are noticeably blurrier in the centre of the linear strip, particularly for DDPM where the effect of a poorly-initialised  $x_T$  compounds over 50 reverse steps. Slerp maintains constant norm throughout the path and avoids this artefact. VQ-VAE uses linear interpolation between codebook embedding maps: two index sequences are sampled from the PixelCNN prior, their embeddings retrieved, and the interpolated maps decoded directly without re-quantisation; occasional abrupt transitions



reflect discrete jumps in the sampled codebook assignments rather than any Gaussian shell effect.

Smooth interpolations across all models indicate that the learned latent spaces suggest that nearby latent codes produce semantically related outputs, though they do not rule out memorisation of training examples. DDPM in  $x_T$  space produces the most photorealistic intermediate frames.

## 7 Mode Collapse Analysis

It is important to distinguish between two related but distinct failure modes observed in the DCGAN training:

**Mode collapse** occurs when the generator maps a large region of the latent space  $\mathcal{N}(0, I)$  to a narrow subset of the data distribution, producing images with limited variety. Symptoms include repeated similar facial proportions and colour palettes across samples drawn from different  $\mathbf{z}$ .

**Generator starvation** is an independent training-budget problem: with  $n_{\text{critic}} = 5$ , the generator receives far fewer weight updates than the discriminator. A poorly-trained generator will score high FID even without collapsing to a single mode, simply because it has not had enough gradient signal to learn the full distribution.

Both effects are present in the original WGAN-GP training run and contribute jointly to the high FID of 221.51. Inspecting generated grids (Figure 5) confirms reduced diversity (mode-collapse symptoms) alongside overall blurriness attributable to insufficient generator updates.

**Mitigation strategies applied:**

- WGAN-GP gradient penalty ( $\lambda = 10$ ) replaces standard discriminator weight clipping, providing smoother gradients to the generator.
- Periodic sample grids saved every 10 epochs were used to monitor for mode collapse during training (no per-epoch FID due to cost).
- Note: the initial implementation contained a label smoothing parameter (real labels  $\rightarrow 0.9$ ) intended for standard BCE-GAN training. This parameter operates on binary cross-entropy targets and has no effect on the WGAN-GP loss path, which uses a scalar critic score rather than binary labels. It was therefore *inactive* during WGAN-GP training.

The generator starvation hypothesis and corrected configurations are analysed quantitatively in Section 8.

## 8 DCGAN Training Bottleneck Analysis

### 8.1 Generator Starvation

A post-hoc analysis of the DCGAN training budget revealed a potential source of under-training. With  $n_{\text{critic}} = 5$  and 156 batches per epoch ( $\lfloor 9997/64 \rfloor$  with `drop_last=True`), the generator receives only  $\lfloor 156/5 \rfloor \times 100 = \mathbf{3\,100}$  update steps over 100 epochs. A

standard DCGAN with  $n_c = 1$  would accumulate  $156 \times 100 = \mathbf{15\,600}$  generator updates at the same computational cost.

## 8.2 Corrected Configurations

Two corrected configurations were trained to test whether increasing the generator update budget improves FID:

1. **Standard DCGAN**: switch to standard BCE loss with  $n_c = 1$  (the canonical DCGAN setting, not a compromise); increased generator capacity ( $g_{\text{ch}} = 128$ ); 150 epochs  $\Rightarrow$  **23 400** G-steps.
2. **WGAN-GP**  $n_c = 2$ : pragmatic reduction of critic steps while retaining the Wasserstein loss; 200 epochs  $\Rightarrow$  **15 600** G-steps. Note that  $n_c < 5$  gives the critic fewer updates per generator step, potentially leaving it less well-optimised; a compute-equivalent run at  $n_c = 5$  would require  $\approx 500$  epochs.

Table 2: FID results for original and corrected DCGAN configurations.

| Configuration                           | G-steps | FID $\downarrow$ | Notes            |
|-----------------------------------------|---------|------------------|------------------|
| Original WGAN-GP ( $n_c = 5$ , 100 ep.) | 3 100   | 221.51           | baseline         |
| Standard DCGAN ( $n_c = 1$ , 150 ep.)   | 23 400  | <b>98.99</b>     | BCE loss         |
| WGAN-GP $n_c = 2$ (200 ep.)             | 15 600  | 158.48           | Wasserstein loss |

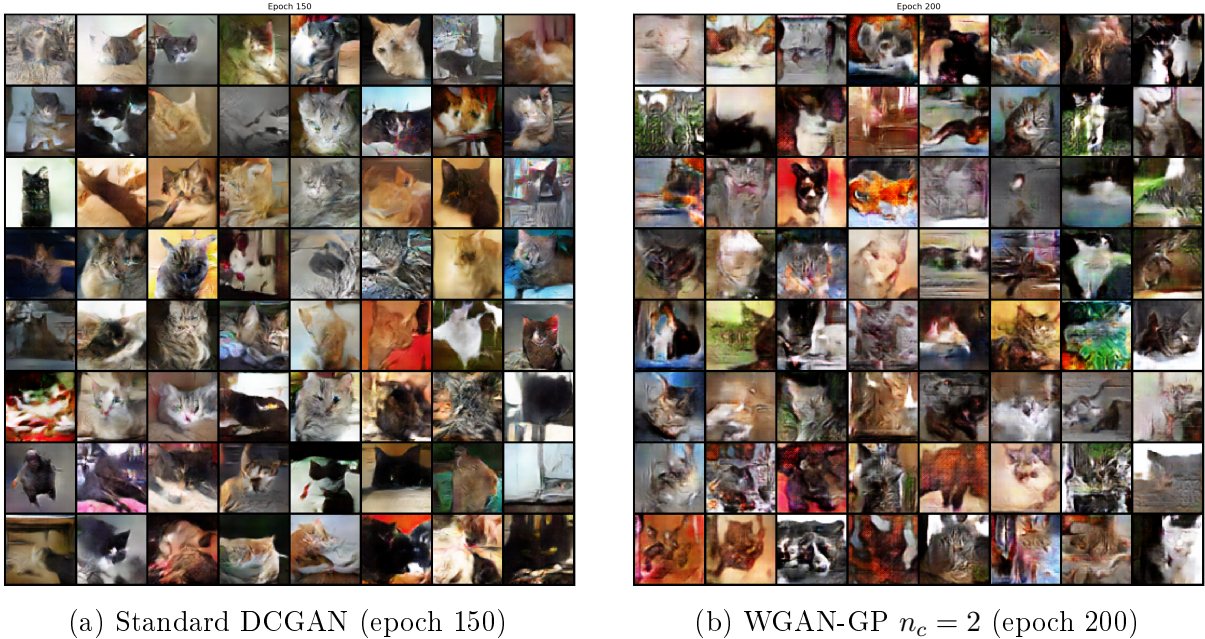


Figure 9: Samples from corrected DCGAN configurations.

## 8.3 Discussion

Both corrected configurations yield substantially lower FID. The standard DCGAN at 23 400 G-steps achieves FID 98.99 in a single run, a  $2.2\times$  improvement over the original. A later 3-seed rerun of the same corrected standard DCGAN configuration (with  $d_{\text{ch}} = 64$ )

produced FID  $142.6 \pm 3.4$ , confirming the improvement over the original but showing that corrected DCGAN remains sensitive to training details and random seed. The WGAN-GP  $n_c = 2$  at 15 600 G-steps achieves FID 158.48.

These results suggest that the original WGAN-GP result was limited not only by architecture but also by the low training budget allocated to generator updates. However, because each corrected configuration changes several factors simultaneously – loss function,  $n_c$ , model capacity, and training duration – these should be interpreted as diagnostic follow-up experiments rather than a clean single-factor ablation. The improvement from standard DCGAN over WGAN-GP  $n_c = 2$  may reflect the larger generator update budget (23 400 vs. 15 600 steps), the switch to BCE loss, or increased generator capacity, and cannot be attributed to any single factor from this experiment alone.

## 9 Ablation Studies

### 9.1 DDPM: DDIM Inference Steps and EMA Weights

We evaluate the effect of the number of DDIM inference steps and EMA weights jointly, using the epoch-200 checkpoint (inference-only, 5 000 samples each).

Table 3: DDPM FID vs. DDIM inference steps and EMA weights (epoch-200 checkpoint, 5 000 samples). Bold marks the default configuration used in Table 1.

| DDIM Steps | FID (raw) ↓ | FID (EMA) ↓  | Speedup |
|------------|-------------|--------------|---------|
| 10         | 66.90       | 30.81        | 100×    |
| 25         | 60.26       | 26.31        | 40×     |
| 50         | 55.43       | <b>24.11</b> | 20×     |
| 100        | 48.35       | 22.52        | 10×     |
| 200        | 39.56       | 21.49        | 5×      |

Raw-weight FID improves monotonically with more steps, with diminishing returns above 100. EMA weights at 50 steps reduce FID from 55.43 to 24.11 (57% reduction), a larger gain than increasing from 50 to 200 steps. The EMA averages weight noise that accumulates during training, which is particularly beneficial for diffusion models because errors in the reverse chain compound over many steps. EMA weights consistently outperform raw weights at the same step count; the largest relative gain is at 50 steps ( $55.43 \rightarrow 24.11$ ), consistent with the main result in Table 1. The 50-step EMA configuration is the default reported throughout this work.

### 9.2 VQ-VAE: Codebook Size

The codebook size  $K$  is the primary hyperparameter governing the expressiveness of the discrete latent space. An initial run with  $K = 1024$  was conducted before settling on the final configuration. With  $K = 1024$ , codebook perplexity collapsed to  $\approx 300/1024$  (29% utilisation) during training, meaning over 70% of codes were never assigned. Switching to  $K = 512$  with EMA codebook updates (decay  $\gamma = 0.99$ ) recovered utilisation to 79% (403/512) and produced visibly sharper reconstructions.

A complementary remedy for codebook collapse is *codebook restart*: dead or under-used codes are re-initialised by sampling from the encoder outputs of the current batch. This technique can recover utilisation without changing  $K$  and is used in several VQ-VAE variants (e.g. [11]). It was not implemented here; halving  $K$  proved sufficient, and restart would add training complexity without a clear advantage given the already high utilisation at  $K = 512$ .

Full multi-seed ablation for VQ-VAE was not conducted due to the 200-epoch (Phase 1 + Phase 2) training cost per run. Other hyperparameters (embedding dimension, commitment cost  $\beta$ , PixelCNN depth) were set based on the original VQ-VAE paper [14] and not systematically swept.

### 9.3 DCGAN: Latent Dimension and Learning Rate

We train DCGAN configurations for 50 epochs each, varying the latent dimension  $z_{\text{dim}}$  and learning rate independently across three seeds per configuration (all other hyperparameters fixed to baseline:  $z_{\text{dim}} = 128$ ,  $\text{lr} = 2 \times 10^{-4}$ ,  $n_c = 5$ ). FID is reported as mean  $\pm$  population standard deviation across seeds.

Table 4: DCGAN ablation: FID (mean  $\pm$  std across 3 seeds, 50 epochs).

| Configuration                                          | FID mean $\pm$ std | $\Delta$ vs. baseline |
|--------------------------------------------------------|--------------------|-----------------------|
| $z_{\text{dim}} = 64$                                  | $246.65 \pm 5.57$  | $-17.60$              |
| $z_{\text{dim}} = 128$ (baseline)                      | $264.25 \pm 20.21$ | $-$                   |
| $z_{\text{dim}} = 256$                                 | $266.05 \pm 2.52$  | $+1.80$               |
| $\text{lr} = 1 \times 10^{-4}$                         | $295.11 \pm 18.27$ | $+30.86$              |
| $\text{lr} = 2 \times 10^{-4}$ (baseline)              | $264.25 \pm 20.21$ | $-$                   |
| $\text{lr} = 4 \times 10^{-4}$                         | $220.05 \pm 4.47$  | $-44.20$              |
| Augmented ( $z_{128}$ , $\text{lr} 2 \times 10^{-4}$ ) | $284.05 \pm 14.80$ | $+19.80$              |

Table 5 lists the individual per-seed FID values underlying the summary statistics.

Table 5: DCGAN ablation: raw FID per seed (seeds 0, 42, 123; 50 epochs each).

| Configuration                             | Seed 0 | Seed 42 | Seed 123 | Mean   | Std   |
|-------------------------------------------|--------|---------|----------|--------|-------|
| $z_{\text{dim}} = 64$                     | 254.40 | 241.55  | 244.01   | 246.65 | 5.57  |
| $z_{\text{dim}} = 128$ (baseline)         | 239.25 | 288.73  | 264.78   | 264.25 | 20.21 |
| $z_{\text{dim}} = 256$                    | 264.20 | 264.33  | 269.61   | 266.05 | 2.52  |
| $\text{lr} = 1 \times 10^{-4}$            | 313.98 | 270.39  | 300.94   | 295.11 | 18.27 |
| $\text{lr} = 2 \times 10^{-4}$ (baseline) | 239.25 | 288.73  | 264.78   | 264.25 | 20.21 |
| $\text{lr} = 4 \times 10^{-4}$            | 226.01 | 215.26  | 218.87   | 220.05 | 4.47  |
| Augmented                                 | 304.45 | 277.92  | 269.78   | 284.05 | 14.80 |

Latent dimension has a small effect: the differences between  $z_{\text{dim}} \in \{64, 128, 256\}$  are within the baseline seed variance ( $\pm 20$ ), so no firm conclusion about the optimal  $z_{\text{dim}}$  can

be drawn. The high baseline variance itself is notable and indicates that DCGAN training outcomes are sensitive to random initialisation.

Learning rate has a much larger impact. A higher lr of  $4 \times 10^{-4}$  reduces mean FID by 44 points with low variance ( $\pm 4.47$ ), while the lowest lr ( $1 \times 10^{-4}$ ) increases mean FID by 31 points. The likely mechanism is that a low learning rate causes the generator to update too slowly to keep pace with the well-trained critic, resulting in poor gradient signal.

Enabling data augmentation hurts FID ( $284.05 \pm 14.80$  vs.  $264.25 \pm 20.21$  baseline). In this experiment, augmentation was applied only to the real images via the DataLoader, not to the generated images before they were passed to the discriminator. As a result, the discriminator was trained to distinguish augmented real images (labelled real) from clean generated images (labelled fake), causing it to accept distortions as a signature of real data and incorrectly steer the generator toward augmentation artefacts rather than the true data distribution. The correct approach – applying identical transforms to both real and generated images before the discriminator – is implemented in strategies such as ADA [8], but was not applied here. This is a known pitfall in GAN augmentation and limits the conclusions that can be drawn from this ablation.

## 10 Cats and Dogs Experiment

### 10.1 Setup

The Kaggle dogs-vs-cats dataset [7] provides 12 500 cat images and 12 500 dog images (25 000 total). We train the same DCGAN architecture (100 epochs, all hyperparameters identical to the cats-only baseline) on the combined dataset without class labels, as an unconditional generative model.

DCGAN was selected for this experiment rather than DDPM for two reasons. First, training speed: DCGAN trains in  $\sim 2$  h vs.  $\sim 12$  h for DDPM, making it feasible to run an exploratory 100-epoch experiment within the available compute budget. Second, the experiment addresses a dataset-composition question (does adding dog images help?) rather than a quality benchmark; the relative change in FID is the key signal, not the absolute value. A higher-quality investigation would replicate this experiment with DDPM, but that would require  $\approx 30$  h of additional compute on the 25 000-image dataset.

### 10.2 Evaluation

FID is computed separately against 5 000 real cat images and 5 000 real dog images to measure how well the generated distribution covers each class.

Table 6: DCGAN (cats+dogs) FID compared to cats-only model.

| Model                      | FID vs. Cats ↓ | FID vs. Dogs ↓ | FID vs. Combined ↓ |
|----------------------------|----------------|----------------|--------------------|
| DCGAN (cats only, 100 ep.) | 221.51         | –              | –                  |
| DCGAN (cats+dogs, 100 ep.) | 206.57         | 197.79         | <b>190.39</b>      |

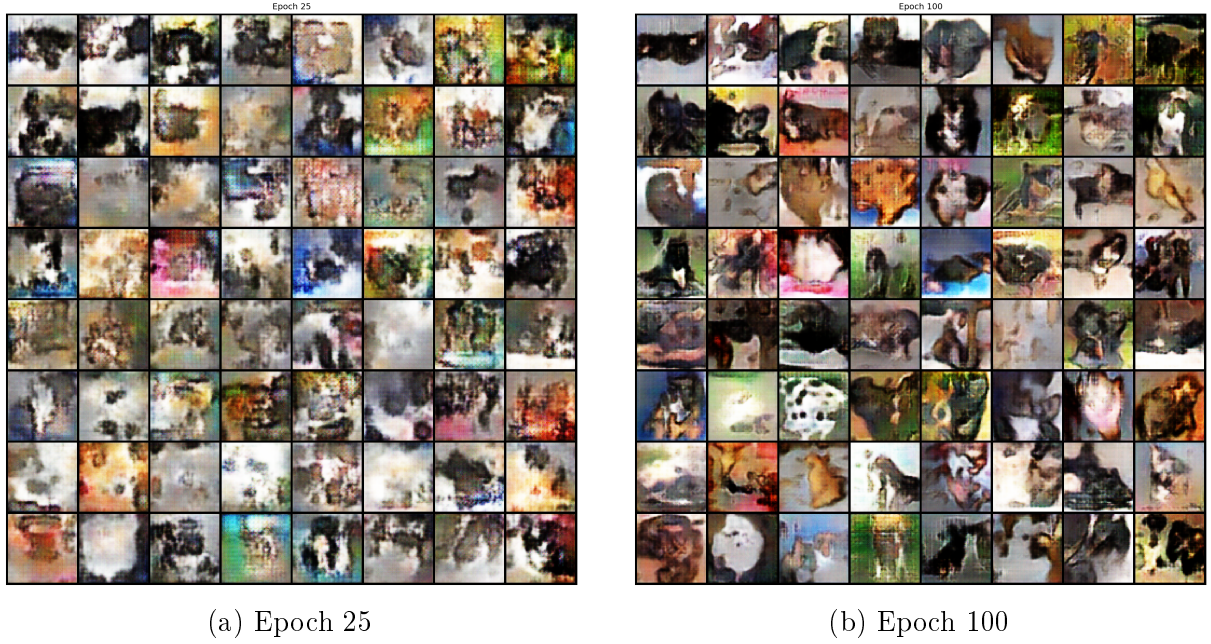


Figure 10: DCGAN (cats+dogs) sample grids at epochs 25 and 100.

### 10.3 Qualitative Assessment

Inspecting the generated samples (Figure 10), the model produces a mixture of recognisable and ambiguous outputs. Some images display distinctly cat-like features: rounded faces, pointed ears, and compact body proportions. Others show dog-like characteristics: longer snouts, floppy ears, and leaner body shapes. However, a large proportion of samples are visually ambiguous – dark quadruped silhouettes without clearly discernible species identity. We observe no systematic collapse to one class: both cat-like and dog-like structures are present throughout the grid. This suggests the unconditional model has learned a blended distribution that captures shared low-level animal features (fur texture, eyes, body outline) while sometimes failing to commit to either species – a predictable consequence of training without class labels on a mixed dataset. A classifier-based analysis (e.g., using a pre-trained cat/dog classifier on the generated samples) would be needed for a quantitative confirmation of this qualitative observation.

### 10.4 Quantitative Discussion

Training an unconditional generative model on a two-class dataset poses an interesting challenge: without conditioning, the model cannot guarantee class-distinct generation.

The cats+dogs DCGAN achieves FID 206.57 against real cats, 15 points lower than the cats-only model’s FID of 221.51. This improvement is suggestive of a beneficial effect: additional dog images may expose the model to a wider variety of animal textures and structures, with shared low-level features (fur, eyes, snout shape) acting as implicit data augmentation. However, this interpretation should be treated cautiously. The 15-point improvement is comparable to the seed variance observed in the DCGAN ablation ( $\sigma \approx 20$  FID), so the result is not statistically conclusive from a single run. Furthermore, FID alone cannot confirm whether the generated images are visually distinct as cats or dogs versus blended representations; a classifier-based analysis would be needed to verify class identity.

in generated samples.

The FID against real dogs (197.79) is similar to the cat FID (206.57), suggesting the model does not strongly collapse to one class.

As a more direct measure of how well the generated distribution covers the training distribution, we also computed FID against a combined reference set of 2500 real cats and 2500 real dogs ( $N = 5000$  total). The resulting FID is 190.39, lower than either individual class FID. This is the most methodologically appropriate comparison for an unconditional model trained on a mixed dataset: the reference set reflects the same class balance as the training data. The improvement over the per-class FIDs (from  $\approx 202$  average to 190) is consistent with the model covering a blended distribution rather than cleanly separating into cat and dog modes.

## 11 Conclusion

We trained and evaluated three families of generative models on cat images:

- **DDPM** achieves the best FID (24.11 with EMA at 50 DDIM steps) and the most visually coherent samples. The 50-step DDIM configuration provides a practical speed-quality trade-off ( $20\times$  speedup over full sampling). Among the inference-time factors tested, EMA weights had the largest effect: they reduce FID by 57% relative to non-EMA inference at the same step count.
- **VQ-VAE + PixelCNN** achieves FID 187.54. Reconstruction quality is good (MSE = 0.0066, codebook utilisation 79%), but the gap between reconstruction quality and sample quality suggests the PixelCNN prior is a likely bottleneck for unconditional generation.
- **DCGAN (WGAN-GP)** achieves FID 221.51, with visible mode collapse symptoms despite WGAN-GP regularisation. Training is the fastest ( $\sim 2$ h). Post-hoc analysis suggests that generator starvation ( $n_c = 5$  yielding only 3 100 G-steps) contributed to the poor result; a corrected standard DCGAN (23 400 G-steps,  $d_{\text{ch}} = 128$ ) achieves FID 98.99 in a single run; a subsequent 3-seed rerun with a smaller discriminator ( $d_{\text{ch}} = 64$ ) gives  $142.6 \pm 3.4$ , and should be treated as a separate configuration. A corrected WGAN-GP ( $n_c = 2$ , 15 600 G-steps) achieves FID 158.48.

These results are consistent with the literature showing that diffusion models offer superior image quality compared to GAN and VAE approaches on small-scale image datasets with limited compute [2].

**Methodological limitations.** The main FID comparisons are single-run results with no standard deviation reported for DDPM and VQ-VAE, because each additional run costs  $\sim 12$ h (DDPM) or  $\sim 4$ h (VQ-VAE). The DCGAN ablation in Section 9.3 was repeated across three seeds to quantify variance; ideally the same rigour would apply to all models. The relative ranking between architectures is reliable for large FID gaps (DDPM vs. the rest) but the VQ-VAE vs. DCGAN difference ( $\approx 34$  FID, or  $\approx 100$  FID after the DCGAN fix) may not be fully robust to seed variation.

The DDPM ablation covers inference-time parameters only (DDIM steps and EMA). Training-time hyperparameters – noise schedule type, number of diffusion steps  $T$ , U-Net channel multipliers, and learning rate – were not systematically ablated; a full grid

search would require multiple retrains.

The comparison is not compute-normalised. No held-out test set was used: FID reflects similarity to the training distribution. The cats-and-dogs improvement and the augmentation penalty are both within the observed DCGAN seed variance ( $\approx \pm 20$  FID) and should therefore be treated as exploratory findings. The corrected DCGAN experiments change multiple factors simultaneously, so the improvement cannot be attributed to generator starvation alone.

Augmentation in the GAN ablation was applied only to real images via the DataLoader, not to generated images before the discriminator – the correct approach for GANs (as implemented in ADA [8]) would apply identical transforms to both. This limits the conclusions of the augmentation ablation.

**DDIM timestep schedule.** The DDIM sampling implementation uses `range(0, T, T//S)` reversed, producing a strided grid starting from  $t = 980$  rather than  $t = T - 1 = 999$  for  $T = 1000$ ,  $S = 50$ . An endpoint-inclusive variant starting at  $t = 999$  was tested post-hoc but produced substantially higher FID ( $\sim 250$  vs.  $\sim 55$  for raw weights at  $S = 50$ ), indicating that the two discretisations act as different sampler variants; the trained checkpoint is evaluated under the same schedule used during development, which is the strided grid. All reported results use this schedule consistently.

**BatchNorm in WGAN-GP critic.** The initial discriminator implementation used `BatchNorm2d`, which mixes batch statistics and is incompatible with the per-sample gradient penalty computation of WGAN-GP. This has been corrected to `InstanceNorm2d (affine=True)`. The corrected WGAN-GP ( $n_c=2$ , 200 epochs) achieves FID 158.48; see Section 8 for details.

**FID reference set.** Real reference images for FID were saved once using a sequential DataLoader (no shuffle) and reused across all evaluations. Since the same fixed set of 5000 images was used for every model comparison, the FID values are internally consistent. The images are drawn from the first 5000 samples in file-sort order across all subdirectories; diversity across subdirectories CAT\_00–CAT\_06 was not explicitly verified but the dataset is known to contain a broad range of cat breeds and poses. FID sample directories are now cleared before each run to prevent accumulation from previous evaluations.

### Potential improvements and future work.

- Training DDPM at  $128 \times 128$  resolution with gradient checkpointing would likely improve FID further.
- A class-conditional DCGAN (e.g., projection discriminator) trained on the cats-and-dogs dataset would enable targeted cat/dog generation and allow a cleaner assessment of multi-class transfer.
- Replacing the PixelCNN prior with a Transformer-based prior (e.g., VQ-VAE-2 [11]) could improve sample diversity.
- A cosine noise schedule for DDPM may improve results at lower resolutions.



## AI Usage Statement

GitHub Copilot / Claude Sonnet was used as a coding assistant for boilerplate generation, debugging PyTorch errors, refactoring, and plotting utilities. AI tools were also used for language polishing and L<sup>A</sup>T<sub>E</sub>X structuring. All experiments were designed, executed, and verified by the author; numerical results were obtained from the implemented scripts. Architectural choices, final experimental design, interpretation of results, and conclusions are the author’s own. Suggested hyperparameter values were cross-checked against standard settings from the cited literature and adjusted to the 6 GiB VRAM constraint (NVIDIA RTX 4050 Laptop GPU).

## Reproducibility

All code is available in the project repository. To reproduce results:

1. Install dependencies: `uv sync` (uses `pyproject.toml`; installs CUDA-enabled PyTorch automatically)
2. Download and place the CAT Dataset in `data/cats/`
3. Train: `uv run train-dcgan`, `uv run train-vqvae`, `uv run train-ddpm`
4. Evaluate: `uv run python evaluate_all.py`
5. Run ablations: `uv run python ablate_ddpm_steps.py` and `uv run python ablate_dcgan_mul`

Global seed 42 is set via `src/utils/seed.py`. All reported FID values are listed in the tables above; the `outputs/` directory is excluded from the repository (large files) but is reproduced by running the evaluation scripts.

## References

- [1] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *International Conference on Machine Learning*, pages 214–223, 2017.
- [2] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat GANs on image synthesis. *Advances in Neural Information Processing Systems*, 34:8780–8794, 2021.
- [3] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, volume 27, 2014.
- [4] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of wasserstein GANs. *Advances in Neural Information Processing Systems*, 30, 2017.
- [5] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. *Advances in Neural Information Processing Systems*, 30, 2017.
- [6] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.

- [7] Kaggle. Dogs vs. cats, 2013. Available at <https://www.kaggle.com/competitions/dogs-vs-cats/>.
- [8] Tero Karras, Miika Aittala, Janne Hellsten, Samuli Laine, Jaakko Lehtinen, and Timo Aila. Training generative adversarial networks with limited data. In *Advances in Neural Information Processing Systems*, volume 33, pages 12104–12114, 2020.
- [9] Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. *International Conference on Learning Representations*, 2014.
- [10] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *International Conference on Learning Representations*, 2016.
- [11] Ali Razavi, Aaron van den Oord, and Oriol Vinyals. Generating diverse high-fidelity images with VQ-VAE-2. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [12] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *International Conference on Learning Representations*, 2021.
- [13] Aaron van den Oord, Nal Kalchbrenner, and Koray Kavukcuoglu. Pixel recurrent neural networks. In *International Conference on Machine Learning*, pages 1747–1756, 2016.
- [14] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [15] Weiwei Zhang, Jian Sun, and Xiaoou Tang. Cat head detection – CAT dataset, 2008. Available at <https://www.kaggle.com/datasets/crawford/cat-dataset>.